



ARTIFICIAL NEURAL NETWORKS (ANN) LAB

10

Ms. Shakeela Shaheen

LAB 10: INTRODUCTION TO LSTM AND IMPLEMENTING LSTM FOR SENTIMENT ANALYSIS

Learning Objectives

By the end of this lab, students will be able to:

1. Explain the limitations of Simple RNNs.
2. Understand why LSTM was introduced.
3. Describe the cell state and the three gates of LSTM.
4. Connect the mathematics of LSTM with its implementation.
5. Prepare text data for LSTM-based sentiment classification.
6. Build, train, and evaluate an LSTM model.
7. Compare LSTM with a simple baseline model.
8. Analyze how parameter changes affect performance.

1. Motivation: Why Do We Need LSTM?

Sequence data often contains information that is not useful immediately, but becomes important later.

Example

Consider this sentence:

“The movie started slowly, but the ending was amazing.”

To understand the overall sentiment, the model must remember:

- “started slowly” → negative tone
- “ending was amazing” → positive tone

A model that forgets early words may give the wrong conclusion.

Problem with Simple RNN

A Simple RNN updates its hidden state step by step, but over long sequences:

- important early information may disappear
- gradients become too small

- learning long-term patterns becomes difficult

That is why RNNs struggle when the sequence is long.

Solution: LSTM

LSTM adds a **memory cell** and **gates** to control what to:

- remember
- forget
- output

This makes LSTM much better at handling long-range dependencies.

2. What is LSTM?

LSTM stands for **Long Short-Term Memory**.

It is an improved type of recurrent neural network that can retain important information for a long time.

Unlike Simple RNN, LSTM does not store everything in one hidden state only.

It has:

- a **cell state** for long-term memory
- a **hidden state** for short-term output
- gates that regulate information flow

3. Intuition Behind LSTM

Think of LSTM like a smart notebook:

- some information is kept for later
- some information is erased
- some information is written now
- some information is shown as output

This selective memory is the main strength of LSTM.

4. LSTM Architecture

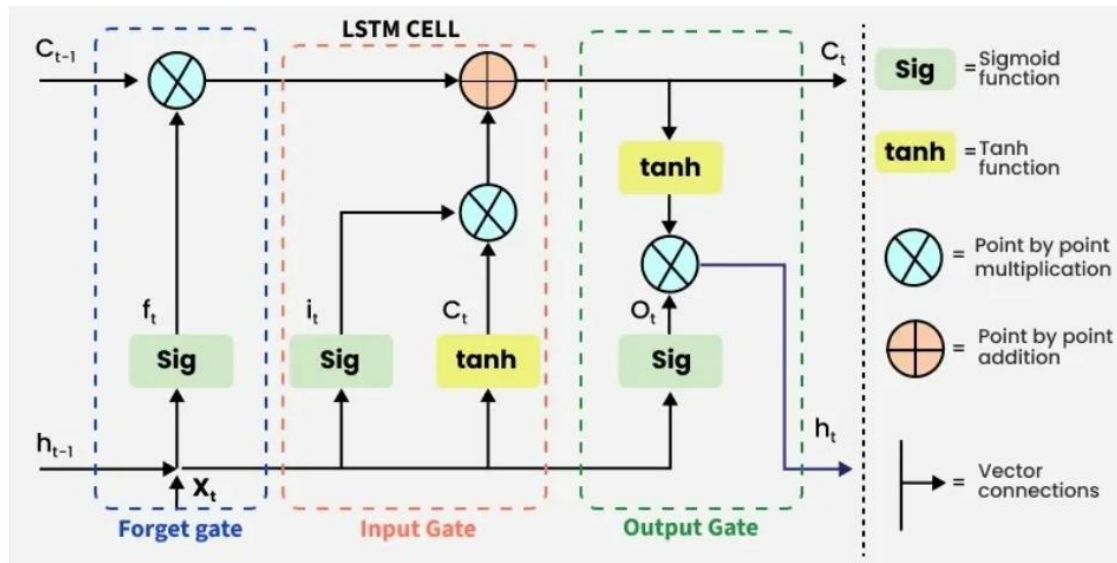
An LSTM cell has three main gates:

1. **Forget Gate**
2. **Input Gate**

3. Output Gate

It also has:

- Cell state C_t
- Hidden state h_t



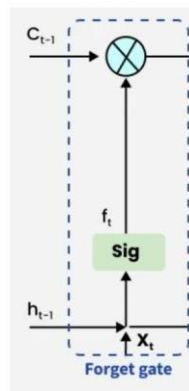
LSTM Model

5. Mathematics of LSTM

Let:

- x_t = input at time step t
- h_{t-1} = previous hidden state
- C_{t-1} = previous cell state

5.1 Forget Gate



Forget Gate

The forget gate decides what information to remove from the previous memory.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Meaning

- σ is the sigmoid function
- output values lie between 0 and 1
- values near 0 mean “forget this”
- values near 1 mean “keep this”

5.2 Input Gate

The input gate decides what new information should be added.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

A candidate memory vector is also created:

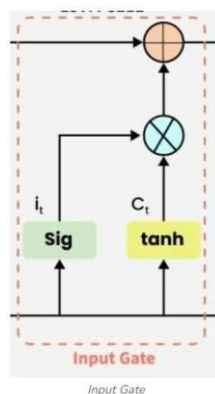
$$C_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Then the cell state is updated:

$$C_t = f_t \odot C_{t-1} + i_t \odot C_t$$

Meaning

- $f_t \odot C_{t-1}$: keep useful old memory
- $i_t \odot C_t$: add new useful memory



5.3 Output Gate

The output gate decides what part of the cell state should be sent forward.

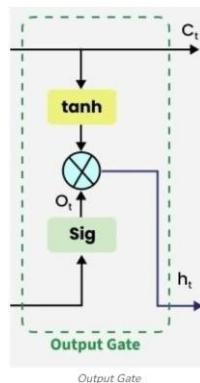
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

The hidden state is then:

$$h_t = o_t \odot \tanh(C_t)$$

Meaning

The hidden state becomes the output of the LSTM cell for that time step.



6. Why LSTM Works Better Than Simple RNN

LSTM helps because:

- it preserves useful memory for longer
- it reduces forgetting of early important words
- gates give control over information flow
- the cell state helps gradients flow better during training

This makes LSTM especially useful for:

- sentiment analysis
- speech recognition
- language modeling
- text classification

7. Full Practical Implementation

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Load or create text data:

```
texts = [
    "I loved this movie",
    "This film was amazing",
    "Absolutely wonderful experience",
    "I hated this movie",
    "The film was boring",
    "Terrible and disappointing"
]

labels = [1, 1, 1, 0, 0, 0]
```

Tokenize the text:

```
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)
print(sequences)
```

Tokenizer converts words into integer IDs.

Example:

“I loved this movie” → [2, 3, 4, 5]

Neural networks cannot process raw words directly, so numbers are required.

Pad sequences:

```
padded_sequences = pad_sequences(sequences, maxlen=6, padding='post', truncating='post')
print(padded_sequences)
```

Different sentences have different lengths.

Padding makes all sequences equal length so the model can process them in batches.

- maxlen=6 means each sentence will be length 6
- shorter sentences get padded with zeros

Convert labels to NumPy array:

```
labels = np.array(labels)
```

Split into training and testing data:

```
split = int(len(padded_sequences) * 0.8)

X_train = padded_sequences[:split]
X_test = padded_sequences[split:]
y_train = labels[:split]
y_test = labels[split:]
```

Build the LSTM model:

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=1000, output_dim=16, input_length=6),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Embedding(...)

This converts each word index into a dense vector representation.

- input_dim=1000 = vocabulary size
- output_dim=16 = embedding size

Why Embedding?

It helps the model learn semantic relationships between words.

LSTM(32)

- 32 memory units
- processes the sequence step by step
- learns temporal dependencies

Dense(1, activation='sigmoid')

- output layer for binary classification
- sigmoid gives a value between 0 and 1

Compile the model:

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

Adam works well in most neural network problems

Binary crossentropy is used for two-class classification

Accuracy measures correct predictions

Train the model:

```
history = model.fit(  
    X_train, y_train,  
    epochs=20,  
    validation_data=(X_test, y_test),  
    verbose=1  
)
```

Evaluate the model:

```
loss, accuracy = model.evaluate(X_test, y_test)  
print("Test Accuracy:", accuracy)
```

Plot training results:

```
plt.figure(figsize=(12, 4))  
  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title("Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.legend()  
  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title("Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.legend()  
  
plt.show()
```

8. Connecting Math to Code:

LSTM Concept	Code Representation
Input sequence	padded_sequences
Word vectors	Embedding(...)
Forget gate	internal LSTM mechanism
Input gate	internal LSTM mechanism
Output gate	internal LSTM mechanism
Hidden state	inside LSTM(32)
Final sentiment prediction	Dense(1, activation='sigmoid')

9. Why Padding and Tokenization Matter

Text has variable length.

Example:

- “Good movie” → 2 words
- “The movie was really amazing and enjoyable” → 7 words

Neural networks need fixed-size input, so:

1. text is tokenized into numbers
2. sequences are padded to equal length

This is a crucial preprocessing step in NLP.

LAB TASK

Scenario

You are hired as a **Junior AI Engineer** at a startup building an **AI-powered Movie Review Analysis System**.

The company collects thousands of user reviews and wants to:

Automatically classify reviews as:

- **Positive**
- **Negative**

However, management reports:

- Traditional models (MLP) are failing to capture context
- Words like “*not good*” are being misclassified as positive
- Model cannot understand long-term dependencies in text

Your task:

Build an **LSTM-based sentiment analysis model** that understands context in sequences.

Dataset (Kaggle)

You can use:

IMDB Dataset of 50K Movie Reviews

Lab Tasks

Task 1 – Data Preprocessing

You must:

- Load dataset (CSV)
- Convert text → sequences using Tokenizer
- Apply **padding** to ensure equal length

Example:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
tokenizer = Tokenizer(num_words=5000)
tokenizer.fit_on_texts(texts)
```

```
sequences = tokenizer.texts_to_sequences(texts)
X = pad_sequences(sequences, maxlen=100)
```

You must explain:

- Why padding is required
- What happens if sequences have different lengths

Task 2 – Build LSTM Model

You must implement:

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=5000, output_dim=64, input_length=100),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Compile:

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

You must explain:

- Role of **Embedding layer**
- Why LSTM is used instead of SimpleRNN
- Why sigmoid is used

Task 3 – Training & Evaluation

Train model:

```
history = model.fit(X_train, y_train,
                    validation_data=(X_val, y_val),
                    epochs=5,
                    batch_size=32)
```

You must:

- Plot training vs validation accuracy

- Identify:
 - Overfitting OR underfitting

Task 4 – Parameter Modification

You must improve model by modifying:

Any TWO:

- LSTM units (32 → 128)
- Add Dropout
- Change sequence length (100 → 200)
- Change optimizer / learning rate

Record:

Model	Train Acc	Val Acc
Original		
Modified		

Task 5 – Prediction Test

Test model on custom inputs:

```
sample = ["This movie was absolutely amazing"]
```

```
sample_seq = tokenizer.texts_to_sequences(sample)
sample_pad = pad_sequences(sample_seq, maxlen=100)
```

```
prediction = model.predict(sample_pad)
```

You must interpret output:

- Close to 1 → Positive
- Close to 0 → Negative

Required Visualizations

You must plot:

- Accuracy vs Epochs
- Loss vs Epochs

Reflection Questions

You must answer:

1. Why does LSTM perform better than SimpleRNN?
2. What role does memory cell play?
3. Why is padding important in NLP tasks?
4. What happens if sequence length is too small?
5. Why does increasing LSTM units improve performance (or not)?